



UNIVERSIDAD TECNOLÓGICA DE PANAMÁ
FACULTAD DE INGENIERÍA DE SISTEMAS
COMPUTACIONALES
LICENCIATURA EN CIBERSEGURIDAD

NOMBRE DE LA ASIGNATURA:

Programación II
“Taller Práctico #1”

PREPARADO POR:

Yisuari Rodríguez, 1-760-982

A CONSIDERACIÓN DE

Napoleón Ibarra

2S3121

10/04/2026

Ventana Principal

```
1 import tkinter as tk
2 from tkinter import ttk, messagebox, filedialog, scrolledtext
3 import json
4 import os
5 from cryptography.fernet import Fernet
6 import base64
7
8 #VENTANA PRINCIPAL
9 class AppPrincipal:
10     def __init__(self, root):
11         self.root = root
12         self.root.title("Sistema de Gestión de Calificaciones y Encriptación")
13         self.root.geometry("800x600")
14         self.root.configure(bg="#f0f4f7")
15
16         # Estilo
17         style = ttk.Style()
18         style.theme_use("clam")
19         style.configure("TButton", font=("Segoe UI", 12, "bold"), padding=10)
20         style.configure("TLabel", font=("Segoe UI", 11), background="#f0f4f7")
21         style.configure("TFrame", background="#f0f4f7")
22
23         # Estilo personalizado para botones grandes
24         style.configure("Big.TButton", font=("Segoe UI", 14, "bold"), padding=20)
25
26         # Frame contenedor principal
27         self.contenedor = ttk.Frame(self.root)
28         self.contenedor.pack(fill="both", expand=True, padx=10, pady=10)
29
30         # Pantalla de bienvenida con botones
31         self.pantalla_inicio()
32
33     def limpiar_contenedor(self):
34         for widget in self.contenedor.winfo_children():
35             widget.destroy()
36
37     def pantalla_inicio(self):
38         self.limpiar_contenedor()
39
40         # Frame centrado para los botones
41         frame_centro = ttk.Frame(self.contenedor)
42         frame_centro.place(relx=0.5, rely=0.5, anchor="center")
43
44         # Título
45         lbl_titulo = ttk.Label(frame_centro,
46                               text="Seleccione una opción",
47                               font=("Segoe UI", 18, "bold"),
48                               background="#f0f4f7")
49
```

```
9 class AppPrincipal:
37     def pantalla_inicio(self):
41         # Frame para los botones
42         frame_botones = ttk.Frame(frame_centro)
43         frame_botones.pack(pady=20)
44
45         # Botón Problema 1
46         btn_problema1 = tk.Button(frame_botones,
47                                   text=" Problema 1\nGestión de Calificaciones",
48                                   font=("Segoe UI", 13, "bold"),
49                                   bg="#3498db",
50                                   fg="white",
51                                   activebackground="#2980b9",
52                                   activeforeground="white",
53                                   relief="flat",
54                                   padx=30,
55                                   pady=20,
56                                   cursor="hand2",
57                                   command=self.abrir_problema1)
58         btn_problema1.pack(side="left", padx=20)
59
60         # Botón Problema 2
61         btn_problema2 = tk.Button(frame_botones,
62                                   text=" Problema 2\nEncriptación de Archivos",
63                                   font=("Segoe UI", 13, "bold"),
64                                   bg="#2ecc71",
65                                   fg="white",
66                                   activebackground="#27ae60",
67                                   activeforeground="white",
68                                   relief="flat",
69                                   padx=30,
70                                   pady=20,
71                                   cursor="hand2",
72                                   command=self.abrir_problema2)
73         btn_problema2.pack(side="left", padx=20)
74
75         # Texto informativo
76         lbl_info = ttk.Label(frame_centro,
77                               text="Seleccione el problema que desea ejecutar",
78                               font=("Segoe UI", 10),
79                               background="#f0f4f7",
80                               foreground="#7f8c8d")
81         lbl_info.pack(pady=20)
82
83     def abrir_problema1(self):
84         self.limpiar_contenedor()
85         Problema1Frame(self.contenedor, self.root)
86
```

```
96         Problema1Frame(self.contenedor, self.root)
97
98     def abrir_problema2(self):
99         self.limpiar_contenedor()
100         Problema2Frame(self.contenedor)
101
```



Problema 1

```
1 # Problema 1 - Sistema de Gestión de Calificaciones (GUI)
2 # Importamos las librerías necesarias
3 import tkinter as tk
4 from tkinter import ttk
5
6 # Se abre ANTES de ProblemaFrame y solicita:
7 # 1. Cantidad máxima de registros (1-128)
8 # 2. Porcentaje de cada criterio de evaluación (suma exacta = 100)
9
10 # Criterios de evaluación
11 CRITERIOS = [
12     "Examen Final",
13     "Exámenes Parciales",
14     "Asignaciones",
15     "Portafolio Digital",
16     "Talleres y Laboratorios",
17     "Asistencia",
18 ]
19
20 # Clase ProblemaFrame
21 class ProblemaFrame(ttk.Frame):
22     def __init__(self, parent=None, root=None, app_principal=None):
23         super().__init__(parent)
24         self.parent_frame = parent
25         self.root = root
26         self.app_principal = app_principal
27
28         # Configuración del Sistema de Calificaciones
29         self.geometry("480x400")
30         self.resizable(False, False)
31         self.grid_rowconfigure(0, weight=1)
32         self.grid_columnconfigure(0, weight=1)
33
34         # Título
35         ttk.Label(self, text="Configuración Inicial",
36                  font=("Sage UI", 14, "bold"),
37                  background="#f0f0f7", padding=(10, 4)).grid(
38             row=0, column=0, rowspan=2,
39             font=("Sage UI", 14, "bold"), background="#f0f0f7",
40             rowspan=2, font=("Sage UI", 14, "bold"), padding=(10, 12))
41
42         # Cantidad de registros
43         frame_cant = ttk.LabelFrame(self, text="Cantidad máxima de registros",
44                                    padding=10)
45         frame_cant.grid(row=0, column=0, rowspan=2,
46                        padding=10)
47         frame_cant.pack(fill="x", padx=24, pady=0)
48
49         # Número de estudiantes a registrar (máx. 128)
50         self.var_cantidad = tk.IntVar(value=30)
51         self.spin_cantidad = ttk.Spinbox(frame_cant, from_=1, to=128, width=40,
52                                         textvariable=self.var_cantidad)
53         self.spin_cantidad.grid(row=0, column=1,
54                                padding=10)
55
56         # Porcentajes de evaluación (deben sumar 100%)
57         frame_porc = ttk.LabelFrame(self, text="Porcentajes de evaluación (deben sumar 100%)",
58                                    padding=10)
59         frame_porc.grid(row=1, column=0, rowspan=2,
60                        padding=10)
61         frame_porc.pack(fill="x", padx=24, pady=0)
62         self.var_porc = {}
63         self.defaults = {
64             "Examen Final": 30,
65             "Exámenes Parciales": 30,
66             "Asignaciones": 10,
67             "Portafolio Digital": 5,
68             "Talleres y Laboratorios": 10,
69             "Asistencia": 5,
70         }
71
72         # Criterio de evaluación
73         for i, criterio in enumerate(CRITERIOS):
74             ttk.Label(frame_porc, text=f"Criterio {i+1}: ",
75                      background="#f0f0f7", padding=(10, 4)).grid(
76                 row=i, column=0, rowspan=2,
77                 var = tk.IntVar(value=self.defaults[criterio])
78             )
79             spin = tk.Spinbox(frame_porc, from_=0, to=100, width=40,
80                             textvariable=self.defaults[criterio])
81             spin.grid(row=i, column=1,
82                      command=self.actualizar_total)
```

```

88 class VentanaConfiguracion(tk.Toplevel):
89     def __init__(self, parent frame, root, app_principal):
90         self.vars_porcentaje = var
91         ttk.Label(frame_porcentaje, text="Total: 100%",
92                  background="#f0f0f0").grid(row=1, column=2, sticky="w")
93         #Indicador de total en tiempo real
94         self.lbl_total = ttk.Label(frame_porcentaje, text="Total: 100%",
95                                   font=("Segoe UI", 18, "bold"),
96                                   foreground="#222222",
97                                   background="#f0f0f0")
98         self.lbl_total.grid(row=len(self.CHATBULOS), column=0,
99                             columnspan=3, pady=(8, 0))
100         #Botones
101         frame_btn = tk.Frame(self)
102         frame_btn.pack(pady=14)
103         tk.Button(frame_btn, text="Confirmar y Continuar",
104                  font=("Segoe UI", 11, "bold"),
105                  bg="#e3e3e3", fg="white", relief="flat",
106                  padx=16, pady=8, cursor="hand2",
107                  activebackground="#d3d3d3", activeforeground="white",
108                  command=self._confirmar).pack(side="left", padx=18)
109         tk.Button(frame_btn, text="Cancelar",
110                  font=("Segoe UI", 11), bg="#e3e3e3", fg="white",
111                  relief="flat", padx=16, pady=8, cursor="hand2",
112                  activebackground="#d3d3d3", activeforeground="white",
113                  command=self._cancelar).pack(side="left", padx=18)
114     def _cancelar(self):
115         self.destroy()
116         self.app_principal.pantalla_inicio()
117     def _actualizar_total(self):
118         try:
119             total = sum(v.get() for v in self.vars_porcentaje.values())
120         except Exception:
121             total = 0
122         color = "#222222" if total == 100 else "#e3e3e3"
123         self.lbl_total.config(text=f"Total: {total}%", foreground=color)
124     def _confirmar(self):
125         # Validar cantidad
126         try:
127             cantidad = int(self.spin_cantidad.get())
128             if not (1 <= cantidad <= 120):
129                 raise ValueError
130         except ValueError:
131             messagebox.showerror("Error",
132                                  "La cantidad debe ser un número entre 1 y 120.", parent=self)
133             return
134         # Validar porcentajes
135         try:
136             porcentajes = {c: int(v.get()) for c, v in self.vars_porcentaje.items()}
137         except Exception:
138             messagebox.showerror("Error",
139                                  "Los porcentajes deben ser números enteros.", parent=self)
140             return
141         total = sum(porcentajes.values())
142         if total != 100:
143             messagebox.showerror("Error",
144                                  "Los porcentajes suman {total}%. Deben sumar exactamente 100%.",
145                                  parent=self)
146             return
147         self.destroy()
148         Problema1Frame(self.parent.frame, self.root.app,
149                        self.app_principal, cantidad, porcentajes)
150 # PROBLEMA 1 - FRAME PRINCIPAL
151 class Problema1Frame:
152     """
153     Dos pestañas:
154     - Pestaña 1 - Registro de Estudiantes y Evaluación
155     - Pestaña 2 - Agregar Estudiantes y Calificaciones
156     """
157     def __init__(self, parent, root, app_principal,
158                  max_registros, max_porcentajes):

```

```

159     """
160     Dos pestañas:
161     - Pestaña 1 - Registro de Estudiantes y Evaluación
162     - Pestaña 2 - Agregar Estudiantes y Calificaciones
163     """
164     def __init__(self, parent, root, app_principal,
165                  max_registros, max_porcentajes):
166         self.parent = parent
167         self.root = root
168         self.app_principal = app_principal
169         self.max_registros = max_registros
170         self.max_porcentajes = max_porcentajes
171         self.archivo_datos = "datos_estudiantes.json"
172         self.datos = self._cargar_datos()
173         # Frame contenedor
174         self.frame_principal = tk.Frame(parent)
175         self.frame_principal.pack(fill="both", expand=True)
176         # Cabeza con botón Home Principal
177         frame_cabecera = tk.Frame(self.frame_principal)
178         frame_cabecera.pack(fill="x", padx=18, pady=(8, 2))
179         tk.Button(frame_cabecera, text="Home Principal",
180                  font=("Segoe UI", 10, "bold"),
181                  bg="#e3e3e3", fg="white", relief="flat",
182                  padx=16, pady=8, cursor="hand2",
183                  activebackground="#d3d3d3", activeforeground="white",
184                  command=self._volver_home).pack(side="left")
185         ttk.Label(frame_cabecera,
186                  text="Sección de Calificaciones - Problema 1",
187                  font=("Segoe UI", 11, "bold")).pack(side="left", padx=14)
188         ttk.Label(frame_cabecera,
189                  text=f"Max. registros: {max_registros}",
190                  font=("Segoe UI", 10), foreground="#e3e3e3").pack(
191                      side="right", padx=4)
192         # Notebook
193         self.notebook = tk.Notebook(self.frame_principal)
194         self.notebook.pack(fill="both", expand=True, padx=4)
195         self._contruir_tab1()
196         self._contruir_tab2()
197     # Pestaña 1 - Registro de Estudiantes y Evaluación
198     def _contruir_tab1(self):
199         tab = tk.Frame(self.notebook)
200         self.notebook.add(tab, text="Registro de Estudiantes y Evaluación")
201         # Resumen de configuración activa
202         frame_config = tk.LabelFrame(tab, text="Configuración activa", padding=4)
203         frame_config.pack(fill="x", padx=18, pady=(8, 4))
204         ttk.Label(frame_config,
205                  text="Capacidad máxima: (self.max_registros) estudiante(s) | "
206                  "Registros actuales: (len(self.datos))",
207                  font=("Segoe UI", 10)).pack(side="left", padding=4)
208         porc_txt = " ".join(f"{c}: {p}%" for c, p in self.max_porcentajes.items())
209         ttk.Label(frame_config, textvariable=porc_txt,
210                  font=("Segoe UI", 10), foreground="#e3e3e3").pack(
211                      side="left", padding=4)
212         # Lista de estudiantes
213         frame_lista = tk.LabelFrame(tab, text="Estudiantes Registrados", padding=4)
214         frame_lista.pack(fill="both", expand=True, padx=18, pady=4)
215         frame_lista = tk.Frame(frame_lista)
216         frame_lista.pack(fill="both", expand=True)
217         columns = ("Nombre", "Apellidos", "Código", "Modalidad", "Nota Final")
218         self.tree = tk.Treview(frame_lista, columns=columns,
219                                show="headings", height=10)
220         ancho = (180, 150, 50, 120, 180)
221         for col, w in zip(columns, ancho):
222             self.tree.heading(col, text=col)
223             self.tree.column(col, width=w, anchor="center")
224         self.tree.configure(scrollcommand=self._scroll)
225         self.tree.pack(side="left", fill="both", expand=True)

```

```

304 # PANTALLA 2 - Agregar Estudiantes y Calificaciones
305
306 def _contruir_tabla(self):
307     tab = ttk.Frame(self.notebook)
308     self.notebook.add(tab, text="Agregar Estudiantes y Calificaciones")
309     # Canvas con scroll para contenido largo
310     canvas = tk.Canvas(tab, bg="#f0f0f0", highlightthickness=0)
311     w = tk.Scrollbar(tab, command=canvas.yview)
312     canvas.config(scrollregion=canvas.bbox())
313     tk.pack(canvas, fill="both", expand=True)
314     canvas.pack(side="left", fill="both", expand=True)
315     inner = ttk.Frame(canvas)
316     w.pack(side="right", fill="y", width=inner, anchor="ne")
317     def _on_configure():
318         canvas.config(scrollregion=canvas.bbox("all"))
319     def _on_canvas_resize(e):
320         canvas.itemconfig(win_id, width=e.width)
321     inner.bind("<Configure>", _on_configure)
322     canvas.bind("<Configure>", _on_canvas_resize)
323     # Datos del estudiante
324     frame_datos = ttk.LabelFrame(inner, text="Datos del Estudiante", padding=10)
325     frame_datos.pack(fill="x", padx=10, pady=(10, 0))
326     campos = {
327         "Nombre": "entry_nombre",
328         "Asignatura": "entry_asignatura",
329         "Grupo": "entry_grupo",
330         "Modalidad (Presencial/Distancia)": "entry_modalidad",
331     }
332     self._entry_nos = {}
333     for i, (campo, key) in enumerate(campos):
334         tk.Label(frame_datos, text=campo).grid(
335             row=i, column=0, sticky="w", padx=5, pady=5)
336         entry = tk.Entry(frame_datos, width=30)
337         entry.grid(row=i, column=1, pady=5, padx=5, sticky="w")
338         self._entry_nos[key] = entry
339
340     frame_btn_datos = ttk.Frame(frame_datos)
341     frame_btn_datos.grid(row=len(campos), column=0, columnspan=2, pady=10)
342     tk.Button(frame_btn_datos, text="Nuevo Estudiante",
343             command=self._agregar_estudiante).pack(side="left", padx=5)
344     tk.Button(frame_btn_datos, text="Limpiar Campos",
345             command=self._limpiar_campos).pack(side="left", padx=5)
346     # Calificaciones
347     frame_notas = ttk.LabelFrame(inner, text="Calificaciones (0-100)", padding=10)
348     frame_notas.pack(fill="x", padx=10, pady=0)
349     self._entry_notas = {}
350     for row, (concepto, pct) in enumerate(self._porcentaje.items()):
351         tk.Label(frame_notas, text=f"{concepto} ({pct}%)").grid(
352             row=row, column=0, sticky="w", padx=5, pady=5)
353         rowspan, columnw, sticky="w", padx=5, pady=5)
354         entry = tk.Entry(frame_notas, width=30)
355         entry.grid(row=row, column=1, padx=5, pady=5)
356         self._entry_notas[concepto] = entry
357
358     self._tbl_resultado = tk.Label(frame_notas, text="",
359             font=("Segoe UI", 12, "bold"), foreground="darkblue")
360     self._tbl_resultado.grid(row=len(self._porcentaje) + 1,
361             column=0, columnspan=2, pady=5)
362
363     frame_btn_notas = ttk.Frame(frame_notas)
364     frame_btn_notas.grid(row=len(self._porcentaje), column=0,
365             columnspan=2, pady=5)
366     tk.Button(frame_btn_notas, text="Calcular Nota Final",
367             command=self._calcular_nota_final).pack(side="left", padx=5)
368     tk.Button(frame_btn_notas, text="Limpiar Notas",
369             command=self._limpiar_notas).pack(side="left", padx=5)
370     tk.Button(inner, text="Ver Registro de Estudiantes",
371             command=self._mostrar_registro).pack(padx=5, pady=5)

```

```

372 # Métodos
373 def _mostrar_registro(self):
374     self.frame_principal.destroy()
375     self.app.principal.pantalla_inicio()
376
377 def _cargar_datos(self):
378     if not self._cargar_datos():
379         try:
380             with open(self._archivo_datos, "r", encoding="utf-8") as f:
381                 return json.load(f)
382         except Exception:
383             return {}
384     return {}
385
386 def _limpiar_datos(self):
387     with open(self._archivo_datos, "w", encoding="utf-8") as f:
388         json.dump(self._datos, f, indent=2, ensure_ascii=False)
389
390 def _limpiar_campos(self):
391     for e in self._entry_nos.values():
392         e.delete(0, tk.END)
393     self._tbl_resultado.config(text="")
394     self._limpiar_notas()
395     for e in self._entry_notas.values():
396         e.delete(0, tk.END)
397     self._tbl_resultado.config(text="")
398
399 def _agregar_estudiante(self):
400     nombre = self._entry_nombre.get().strip()
401     asignatura = self._entry_asignatura.get().strip()
402     grupo = self._entry_grupo.get().strip()
403     modalidad = self._entry_modalidad.get().strip()
404     if not all([nombre, asignatura, grupo, modalidad]):
405         messagebox.showerror("Error", "Todos los campos son obligatorios.")
406         return
407     if len(self._datos) >= self._max_registros:
408         messagebox.showerror("Lista alcanzada",
409             "Ya se alcanzó el número de (self._max_registros) registros(s).")
410         return
411     key = f"{nombre}_{asignatura}_{grupo}"
412     if key in self._datos:
413         if not messagebox.askyesno("Confirmar",
414             "Ya existe un estudiante con ese nombre/asignatura/grupo. ¿"
415             "Desea actualizar sus datos?"):
416             return
417     self._datos[key] = {
418         "nombre": nombre, "asignatura": asignatura,
419         "grupo": grupo, "modalidad": modalidad,
420         "nota": 0, "nota_final": None,
421     }
422     self._guardar_datos()
423     self._actualizar_tabla()
424     self._limpiar_campos()
425     messagebox.showinfo("Éxito", "Estudiante guardado correctamente.")
426
427 def _calcular_nota_final(self):
428     seleccion = self._tbl_resultado.selection()
429     if not seleccion:
430         messagebox.showerror("Error",
431             "Selecciona un estudiante en la pestaña de Registro primero.")
432         return
433     valores = self._tbl_resultado.selection()[0].values()
434     nombre, asignatura, grupo = valores[0], valores[1], valores[2]
435     key = f"{nombre}_{asignatura}_{grupo}"
436     notas = []
437     suma = 0.0
438     try:
439         for concepto, pct in self._porcentaje.items():
440             val = self._entry_notas[concepto].get().strip()
441             if not val:

```

```

404     messagebox.showerror("Error de valor", str(e))
405     return
406     self.datos[key]["notas"] = notas
407     self.datos[key]["nota_final"] = suma
408     self.guardar_datos()
409     self.lbl_resultado.config(
410         text=f"Nota final: (suma: {sum(notas)})",
411         foreground="red" if suma >= 60 else "black"
412     )
413     self.actualizar_tree()
414     messagebox.showinfo("Titulo", f"Nota final de (nombre): (suma: {sum(notas)})")
415     self.eliminar_estudiante(self):
416     sel = self.tree.selection()
417     if not sel:
418         messagebox.showerror("Error", "Selecciona un estudiante para eliminar.")
419     return
420     sel = self.tree.item(sel[0])["values"]
421     key = f"{sel[0]}_{sel[1]}_{sel[2]}"
422     if messagebox.askyesno("Confirmar", f"Eliminar a '{sel[0]}'?"):
423         del self.datos[key]
424         self.guardar_datos()
425         self.actualizar_tree()
426         self.limpiar_evento()
427         self.lbl_resultado.config(text="")
428     def actualizar_tree(self):
429         for item in self.tree.get_children():
430             self.tree.delete(item)
431         for data in self.datos.values():
432             nf = data.get("nota_final")
433             txt = f"({nf:.2f})" if nf is not None else "Pendiente"
434             self.tree.insert("", "end", values=[
435                 data["nombre"], data["asignatura"],
436                 data["grupo"], data["modalidad"], txt])
437     def seleccionar_en_tabl(self, event):
438         """Al seleccionar en tabl carga los datos en tabl automáticamente."""
439         sel = self.tree.selection()
440         if not sel:
441             return
442         vals = self.tree.item(sel[0])["values"]
443         nombre, asignatura, grupo, modalidad, _ = vals
444         key = f"{nombre}_{asignatura}_{grupo}"
445         self.entradas["entry_nombre"].delete(0, tk.END)
446         self.entradas["entry_nombre"].insert(0, nombre)
447         self.entradas["entry_asignatura"].delete(0, tk.END)
448         self.entradas["entry_asignatura"].insert(0, asignatura)
449         self.entradas["entry_grupo"].delete(0, tk.END)
450         self.entradas["entry_grupo"].insert(0, grupo)
451         self.entradas["entry_modalidad"].delete(0, tk.END)
452         self.entradas["entry_modalidad"].insert(0, modalidad)
453         self.limpiar_notas()
454         notas_guardadas = self.datos[key].get("notas", [])
455         for concepto, entry in self.entradas.items():
456             if concepto in notas_guardadas:
457                 entry.insert(0, str(notas_guardadas[concepto]))
458         nf = self.datos[key].get("nota_final")
459         if nf is not None:
460             self.lbl_resultado.config(
461                 text=f"Nota final: (nf: {nf:.2f})",
462                 foreground="red" if nf >= 60 else "black"
463             )
464     def ver_listado(self):
465         ventana = tk.Toplevel(self.root)
466         ventana.title("Listado Completo de Calificaciones")
467         ventana.geometry("780x580")
468         texto = scrolledtext.ScrolledText(ventana, width=90, height=18,
469             font=("Consolas", 10))
470         texto.pack(padx=10, pady=10, fill="both", expand=True)
471         linea = "-" * 72
472         contenido = f"{linea}\n"
473         contenido += "LISTADO COMPLETO DE CALIFICACIONES\n"

```


Configuración del Sistema de Calificaciones

Configuración Inicial

Complete los datos antes de continuar

Cantidad máxima de registros

Número de estudiantes a registrar (máx. 120):

Porcentajes de evaluación (deben sumar 100%)

Examen Final:	33	%
Exámenes Parciales:	30	%
Asignaciones:	10	%
Portafolio Digital:	5	%
Talleres y Laboratorios:	17	%
Asistencia:	5	%
Total: 100%		

Confirmar y Continuar

Cancelar

— Menú Principal

Gestión de Calificaciones — Problema 1

Máx. registros: 30

Registro de Estudiantes y Evaluación

Agregar Estudiantes y Calificaciones

Datos del Estudiante

Nombre:

Asignatura:

Grupo:

Modalidad (Presencial/Distancia):

Guardar Estudiante

Limpiar Campos

Calificaciones (0 — 100)

Examen Final (33%):

Exámenes Parciales (30%):

Asignaciones (10%):

Portafolio Digital (5%):

Talleres y Laboratorios (17%):

Asistencia (5%):

Calcular Nota Final

Limpiar Notas

[illegible]

```

300         text="Clase PÚBLICA para encriptar" : Clase PRIVADA para desencriptar",
301         text="Depos 10", 0, foreground="red"), pack(padx=10, 1))
302
303     # Generación del par de claves
304     from_gen = tit.labelFrame(tab, text="Generar Par de Claves RSA", padding=10)
305     from_gen.pack(fill="x", padx=10, pady=5)
306     tit.button(from_gen, text="Generar Par de Claves (publica + privada)",
307               command=self._ai_generar_par, pack(side="left", padx=5))
308     self.lbl_par = tit.label(from_gen, text="No se han generado claves.", foreground="gray")
309     self.lbl_par.pack(side="left", padx=5)
310
311     # Carga Clases Individualmente
312     from_cargar = tit.labelFrame(tab, text="Cargar Clases Existentes", padding=10)
313     from_cargar.pack(fill="x", padx=10, pady=5)
314     tit.button(from_cargar, text="Cargar Clase Pública (.pub)",
315               command=self._ai_cargar_publica, pack(side="left", padx=5))
316     self.lbl_pub = tit.label(from_cargar, text="Sin cargar", foreground="gray")
317     self.lbl_pub.pack(side="left", padx=5)
318     tit.button(from_cargar, text="Cargar Clase Privada (.pem)",
319               command=self._ai_cargar_privada, pack(side="left", padx=5))
320     self.lbl_priv = tit.label(from_cargar, text="Sin cargar", foreground="gray")
321     self.lbl_priv.pack(side="left", padx=5)
322
323     # Acciones
324     from_act = tit.labelFrame(tab, text="Acciones", padding=10)
325     from_act.pack(fill="x", padx=10, pady=5)
326     tit.button(from_act, text="Encriptar con Clase Pública",
327               command=self._ai_encriptar, pack(side="left", padx=5))
328     tit.button(from_act, text="Desencriptar con Clase Privada",
329               command=self._ai_desencriptar, pack(side="left", padx=5))
330     self.lbl_estado_act = tit.label(from_act, text="", foreground="blue")
331     self.lbl_estado_act.pack(side="left", padx=10)
332
333     # METODOS - Menu
334     def _volver_menu(self):
335         self.frame_principal.destroy()
336         self._app_principal.start(self)
337
338     # METODOS - Clases RSA
339     def _ai_generar_clave(self):
340         try:
341             self.clave_sintetica = random.randrange(2**1024)
342             archivo = filedialog.askopenfilename(
343                 defaultextension=".key",
344                 filetypes=[("key files", "*.key")])
345             if archivo:
346                 with open(archivo, "ab") as f:
347                     f.write(self.clave_sintetica)
348                 self.lbl_estado_act.config(
349                     text="Clave guardada: (%s.path.basename(archivo))",
350                     foreground="green")
351                 messagebox.showinfo("Éxito", "Clave sintética generada y guardada.")
352             except Exception as e:
353                 messagebox.showerror("Error", "No se pudo generar la clave: (%s)" % e)
354
355         def _ai_cargar_clave(self):
356             try:
357                 archivo = filedialog.askopenfilename(
358                     filetypes=[("key files", "*.key")])
359                 if archivo:
360                     with open(archivo, "rb") as f:
361                         self.clave_sintetica = f.read()
362                         self.lbl_estado_act.config(
363                             text="Clave cargada: (%s.path.basename(archivo))",
364                             foreground="green")
365                         messagebox.showinfo("Éxito", "Clave sintética cargada.")
366             except Exception as e:
367                 messagebox.showerror("Error", "No se pudo cargar la clave: (%s)" % e)
368
369         def _ai_encriptar(self):
370             try:
371                 if self.clave_sintetica is None:
372                     messagebox.showerror("Error", "Primero genere o cargue una clave sintética.")
373                 return
374                 archivo_origen = filedialog.askopenfilename(
375                     title="Seleccione archivo a encriptar",
376                     filetypes=[("Archivos encriptados", "*.encrypted")])
377                 if not archivo_origen:
378                     return
379                 with open(archivo_origen, "rb") as f:
380                     datos_enc = f.read()
381             except Exception as e:
382                 return
383             try:
384                 datos_dec = random(self.clave_sintetica).decrypt(datos_enc)
385             except Exception as e:
386                 messagebox.showerror("Error", "La clave no es válida para este archivo.")
387             return
388             archivo_destino = filedialog.askopenfilename(
389                 title="Guardar archivo desencriptado como...",
390                 filetypes=[("Archivos desencriptados", "*.dec")])
391             if not archivo_destino:
392                 return
393             with open(archivo_destino, "wb") as f:
394                 f.write(datos_dec)
395             self.lbl_estado_act.config(
396                 text="Desencriptado exitosamente", foreground="green")
397             messagebox.showinfo("Éxito", "Archivo desencriptado correctamente.")
398             except Exception as e:
399                 messagebox.showerror("Error", "Error al desencriptar: (%s)" % e)
400
401     # METODOS - Algoritmo (RSA)
402     def _ai_generar_par(self):
403         """Genera un par de claves RSA y pide al usuario dónde guardarlas."""
404         try:
405             from cryptography.hazmat.primitives.asymmetric import rsa
406             from cryptography.hazmat.primitives import serialization
407
408             clave_privada = rsa.generate_private_key(
409                 public_exponent=65537,
410                 key_size=2048)
411             clave_publica = clave_privada.public_key()
412
413             # Guardar clave privada
414             arch_priv = filedialog.askopenfilename(
415                 title="Guardar clave PRIVADA como...",
416                 defaultextension=".pem",
417                 filetypes=[("key files", "*.pem")])
418             if not arch_priv:
419                 return
420             with open(arch_priv, "wb") as f:
421                 f.write(clave_privada.private_bytes(
422                     encoding=serialization.Encoding.PEM,
423                     format=serialization.PrivateFormat.TraditionalOpenSSL,
424                     encryption_algorithm=serialization.NoEncryption))
425             # Guardar clave pública
426             arch_pub = filedialog.askopenfilename(
427                 title="Guardar clave PÚBLICA como...",
428                 defaultextension=".pub",
429                 filetypes=[("key files", "*.pub")])
430             if not arch_pub:
431                 return
432             with open(arch_pub, "wb") as f:
433                 f.write(clave_publica.public_bytes(
434                     encoding=serialization.Encoding.PEM,
435                     format=serialization.PublicFormat.SubjectPublicKeyInfo))
436
437             # Cargar en memoria para uso inmediato
438             self.clave_privada = clave_privada

```

```

439
440         def _ai_desencriptar(self):
441             try:
442                 if self.clave_sintetica is None:
443                     messagebox.showerror("Error", "Primero genere o cargue una clave sintética.")
444                 return
445                 archivo_origen = filedialog.askopenfilename(
446                     title="Seleccione archivo a desencriptar",
447                     filetypes=[("Archivos encriptados", "*.encrypted")])
448                 if not archivo_origen:
449                     return
450                 with open(archivo_origen, "rb") as f:
451                     datos_enc = f.read()
452             except Exception as e:
453                 return
454             try:
455                 datos_dec = random(self.clave_sintetica).decrypt(datos_enc)
456             except Exception as e:
457                 messagebox.showerror("Error", "La clave no es válida para este archivo.")
458             return
459             archivo_destino = filedialog.askopenfilename(
460                 title="Guardar archivo desencriptado como...",
461                 filetypes=[("Archivos desencriptados", "*.dec")])
462             if not archivo_destino:
463                 return
464             with open(archivo_destino, "wb") as f:
465                 f.write(datos_dec)
466             self.lbl_estado_act.config(
467                 text="Desencriptado exitosamente", foreground="green")
468             messagebox.showinfo("Éxito", "Archivo desencriptado correctamente.")
469             except Exception as e:
470                 messagebox.showerror("Error", "Error al desencriptar: (%s)" % e)
471
472     # METODOS - Algoritmo (RSA)
473     def _ai_generar_par(self):
474         """Genera un par de claves RSA y pide al usuario dónde guardarlas."""
475         try:
476             from cryptography.hazmat.primitives.asymmetric import rsa
477             from cryptography.hazmat.primitives import serialization
478
479             clave_privada = rsa.generate_private_key(
480                 public_exponent=65537,
481                 key_size=2048)
482             clave_publica = clave_privada.public_key()
483
484             # Guardar clave privada
485             arch_priv = filedialog.askopenfilename(
486                 title="Guardar clave PRIVADA como...",
487                 defaultextension=".pem",
488                 filetypes=[("key files", "*.pem")])
489             if not arch_priv:
490                 return
491             with open(arch_priv, "wb") as f:
492                 f.write(clave_privada.private_bytes(
493                     encoding=serialization.Encoding.PEM,
494                     format=serialization.PrivateFormat.TraditionalOpenSSL,
495                     encryption_algorithm=serialization.NoEncryption))
496             # Guardar clave pública
497             arch_pub = filedialog.askopenfilename(
498                 title="Guardar clave PÚBLICA como...",
499                 defaultextension=".pub",
500                 filetypes=[("key files", "*.pub")])
501             if not arch_pub:
502                 return
503             with open(arch_pub, "wb") as f:
504                 f.write(clave_publica.public_bytes(
505                     encoding=serialization.Encoding.PEM,
506                     format=serialization.PublicFormat.SubjectPublicKeyInfo))
507
508             # Cargar en memoria para uso inmediato
509             self.clave_privada = clave_privada

```



```

768 # cargar en memoria para ser eliminado
769 self.clave_privada = clave_privada
770 self.clave_publica = clave_publica
771 self.lln_priv.config()
772 test= """Por generados (%s.path.hassome/arch_priv/) (%s.path.hassome/foreground/green)"""
773 self.lln_priv.config()
774 self.lln_priv.config(%(path), foreground='green')
775 self.lln_priv.config()
776 test="""%s.path.hassome/arch_priv/, foreground='green'"""
777 mensaje.showInfo("Exitto")
778 "Por la claves tra generadas:\n"
779 "Privada = (%s.path.hassome/arch_priv)\n"
780 "Publica = (%s.path.hassome/arch_pub)\n"
781 except Exception as e:
782     mensaje.showError("Error", "No se pudo generar el par: (%e)")
783 def _sal_cargar_publica(self):
784     try:
785         from cryptography.hazmat.primitives import serialization
786         archivo = os.listdir(os.path.dirname(__file__))
787         title="Cargar clave publica."
788         hintname=["PROM public key", "-pub"], ["All files", "-*.*"]
789         if archivo:
790             with open(archivo, "rb") as f:
791                 self.clave_publica = serialization.load_public_key(f.read())
792             self.lln_priv.config()
793             self.lln_priv.config(%(path), foreground='green')
794             mensaje.showInfo("Exitto", "Clave publica cargada.")
795         except Exception as e:
796             mensaje.showError("Error", "No se pudo cargar la clave publica: (%e)")
797     def _sal_cargar_privada(self):
798         try:
799             from cryptography.hazmat.primitives import serialization
800             archivo = os.listdir(os.path.dirname(__file__))
801             title="Cargar clave privada."
802             hintname=["PROM private key", "-pem"], ["All files", "-*.*"]
803             if archivo:
804                 with open(archivo, "rb") as f:
805                     self.clave_privada = serialization.load_private_key(
806                         f.read(), password=None)
807                 self.lln_priv.config()
808                 self.lln_priv.config(%(path), foreground='green')
809                 mensaje.showInfo("Exitto", "Clave privada cargada.")
810             except Exception as e:
811                 mensaje.showError("Error", "No se pudo cargar la clave privada: (%e)")
812         def _al_encryptar(self):
813             """Encripta con clave publica (RSA-OAEP)"""
814             from cryptography.hazmat.primitives.asymmetric import padding
815             from cryptography.hazmat.primitives import hashes
816             if self.clave_publica is None:
817                 mensaje.showError("Error",
818                                     "Primero genera o carga una clave publica.")
819             return
820             archio_origin = os.listdir(os.path.dirname(__file__))
821             title="Seleccionar archivo a encriptar)"
822             if not archio_origin:
823                 return
824             with open(archivo_origin, "rb") as f:
825                 datos = f.read()
826                 if len(datos) > 100:
827                     mensaje.showError("Error",
828                                         "El archivo pasa limit de bytes.")
829                     "Se solo soporta archivos de hasta -100 bytes."
830                     "Solo la palabra Simétrica para archivos grandes."
831             datos_encl = self.clave_publica.encrypt(

```

```

825     archivo_destino = #FileDialog.askopenfilename(
826     defaultextension=".rsa",
827     filetypes=[("RSA encriptado", "*.rsa")])
828
829     if archivo_destino:
830         with open(archivo_destino, "wb") as f:
831             f.write(datos_enc)
832             self.lbl_estado.asi.config(
833                 text="Encriptado con clave pública", foreground="green")
834             messagebox.showinfo("Éxito", "Archivo encriptado con RSA correctamente.")
835     except Exception as e:
836         messagebox.showerror("Error", f"Error al encriptar: {e}")
837
838 def asi_descryptar(self):
839     """Desencripta con clave privada (RSA-OAEP)"""
840     try:
841         from cryptography.hazmat.primitives.asymmetric import padding
842         from cryptography.hazmat.primitives import hashes
843
844         if self.clave_privada is None:
845             messagebox.showerror("Error",
846                 "Primero genera o carga una clave privada.")
847         return
848
849         archivo_origen = #FileDialog.askopenfilename(
850             title="Seleccionar archivo a desencriptar",
851             filetypes=[("RSA encriptado", "*.rsa"), ("All Files", "*.*")])
852         if not archivo_origen:
853             return
854
855         with open(archivo_origen, "rb") as f:
856             datos_enc = f.read()
857
858         try:
859             datos_dec = self.clave_privada.decrypt(
860                 datos_enc,
861                 padding.OAEP(
862                     mgf=padding.MGF1(algorithm=hashes.SHA256()),
863                     algorithm=hashes.SHA256(),
864                     label=None
865                 )
866             )
867         except Exception:
868             messagebox.showerror("Error",
869                 "La clave privada no corresponde a este archivo.")
870         return
871
872         archivo_destino = #FileDialog.askopenfilename(
873             title="Guardar archivo desencriptado com...")
874         if archivo_destino:
875             with open(archivo_destino, "wb") as f:
876                 f.write(datos_dec)
877             self.lbl_estado.asi.config(
878                 text="(Desencriptado con clave privada", foreground="green")
879             messagebox.showinfo("Éxito", "Archivo desencriptado con RSA correctamente.")
880     except Exception as e:
881         messagebox.showerror("Error", f"Error al desencriptar: {e}")
882
883 # EJECUTAR LA APLICACIÓN
884 if __name__ == "__main__":
885     root = Tk()
886     app = Aplicacion(root)
887     root.mainloop()

```

SimétricaAsimétrica

Clave PÚBLICA para encriptar · Clave PRIVADA para desencriptar.

Generar Par de Claves RSA

Generar Par de Claves (pública + privada)

No se han generado claves.

Cargar Claves Existentes

Cargar Clave Pública (.pub)

Sin cargar

Cargar Clave Privada (.pem)

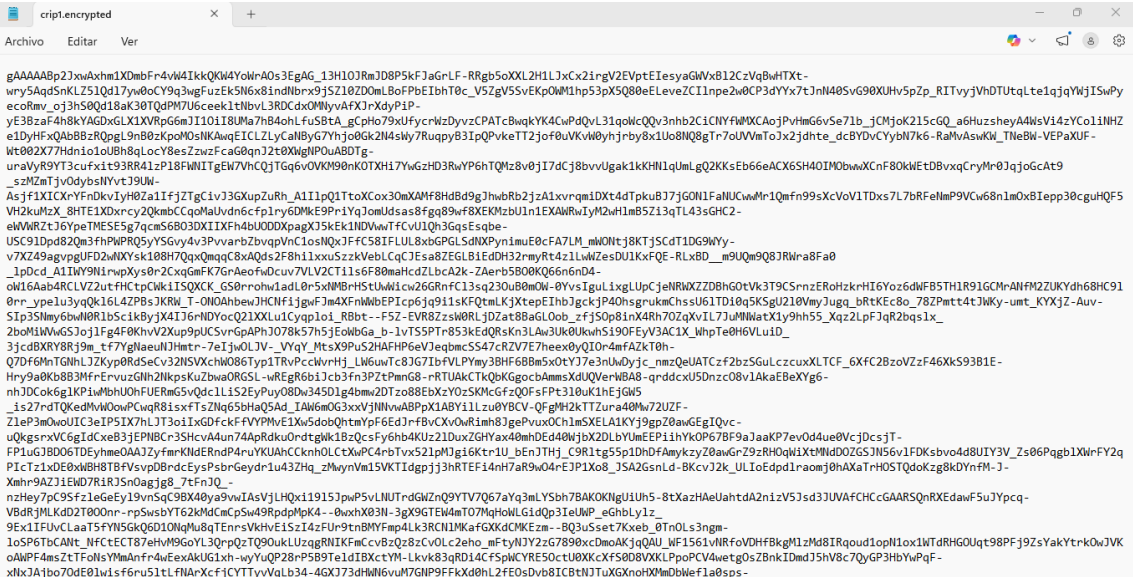
Sin cargar

Acciones

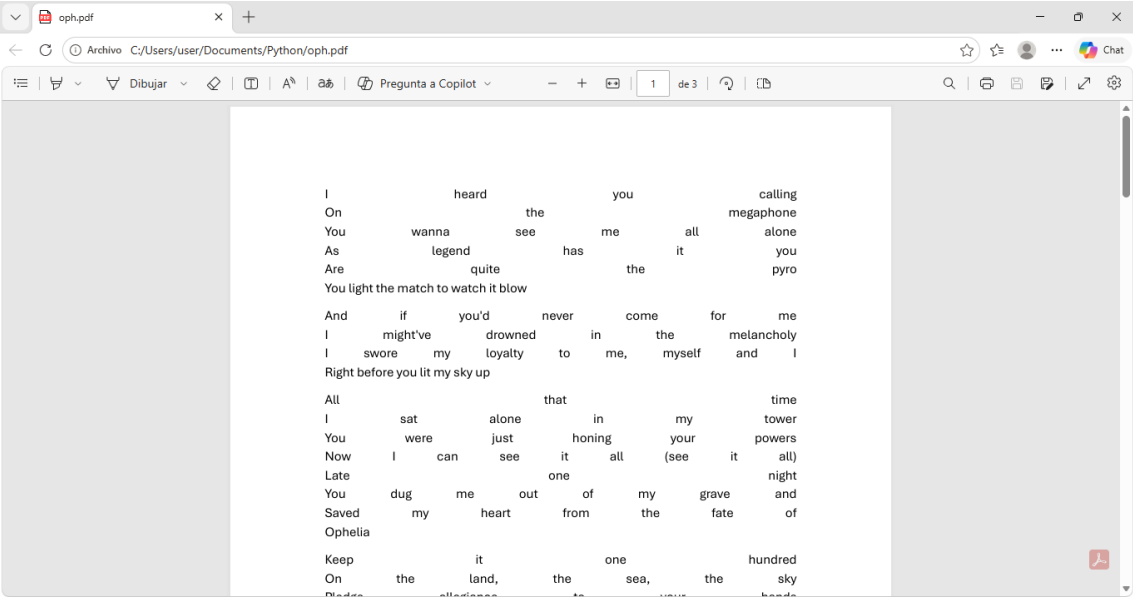
Encriptar con Clave Pública

Desencriptar con Clave Privada

Archivo encriptado



Archivo desencriptado



Laboratorio

Primero verificamos que Python y PiP estén instalados en Kali.

```
(kali@kali)-[~]  
$ python3 --version  
Python 3.13.12  
  
File System  
  
(kali@kali)-[~]  
$  
  
(kali@kali)-[~]  
$ pip3 --version  
pip 26.0.1 from /usr/lib/python3/dist-packages/pip (python 3.13)
```

Luego descargamos cryptography y tkinter.

```
(root@kali)-[/home/kali]  
# apt install python3-cryptography  
python3-cryptography is already the newest version (46.0.5-2).  
python3-cryptography set to manually installed.  
The following packages were automatically installed and are no longer required:  
libdnsl3.6 libicu76 libonnxruntime-providers libonnxruntime1.21 libxnnpack0.20241108 libzxing3 python3-pysnmp4  
Use 'sudo apt autoremove' to remove them.  
  
Summary:  
Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 165  
  
(root@kali)-[/home/kali]  
# apt install python3-tk  
  
python3-tk is already the newest version (3.13.9-3).  
python3-tk set to manually installed.  
The following packages were automatically installed and are no longer required:  
libdnsl3.6 libicu76 libonnxruntime-providers libonnxruntime1.21 libxnnpack0.20241108 libzxing3 python3-pysnmp4  
Use 'sudo apt autoremove' to remove them.  
  
Summary:  
Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 165
```

Ahora crearemos un directorio para el proyecto.

```
(kali㉿kali)-[~]
$ mkdir ~/lab_programacion

(kali㉿kali)-[~]
$ \
Desktop/ Documents/ Downloads/ lab_programacion/ Music/ Pictures/ Public/ Templates/ Videos/

(kali㉿kali)-[~]
$ cd lab_programacion

(kali㉿kali)-[~/lab_programacion]
$
```

Creamos un archivo con nano y pegamos nuestro código.

```
(kali㉿kali)-[~/lab_programacion]
$ nano programa_pro.py

GNU nano 8.7.1 programa_pro.py *
import tkinter as tk
from tkinter import ttk, messagebox, filedialog, scrolledtext
import json
import os
from cryptography.fernet import Fernet
import base64

#VENTANA PRINCIPAL
class AppPrincipal:
    def __init__(self, root):
        self.root = root
        self.root.title("Sistema de Gestión de Calificaciones y Encriptación")
        self.root.geometry("800x600")
        self.root.configure(bg="#f0f4f7")

        # Estilo
        style = ttk.Style()
        style.theme_use("clam")
        style.configure("TButton", font=("Segoe UI", 12, "bold"), padding=10)
        style.configure("TLabel", font=("Segoe UI", 11), background="#f0f4f7")
        style.configure("TFrame", background="#f0f4f7")

        # Estilo personalizado para botones grandes
        style.configure("Big.TButton", font=("Segoe UI", 14, "bold"), padding=20)

        # Frame contenedor principal
        self.contenedor = ttk.Frame(self.root)
        self.contenedor.pack(fill="both", expand=True, padx=10, pady=10)

[...]
```

Ahora crearemos el archivo del script.

```
(kali㉿kali)-[~/lab_programacion]
$ nano ejecutar_lab.sh
```

Lo siguiente será configurar los permisos del script y probaremos el script manualmente

```
(kali㉿kali)-[~/lab_programacion]
$ chmod +x ejecutar_lab.sh

(kali㉿kali)-[~/lab_programacion]
$ ls -la ejecutar_lab.sh
-rwxrwxr-x 1 kali kali 1668 Apr 10 04:05 ejecutar_lab.sh

(kali㉿kali)-[~/lab_programacion]
$ chmod 755 ~/lab_programacion
```

```
(kali㉿kali)-[~/lab_programacion]
$ ls
ejecucion.log  ejecutar_lab.sh  programa_pro.py

(kali㉿kali)-[~/lab_programacion]
$ nano ejecutar_lab.sh

(kali㉿kali)-[~/lab_programacion]
$ chmod +x ejecutar_lab.sh

(kali㉿kali)-[~/lab_programacion]
$ chmod 755 ~/lab_programacion

(kali㉿kali)-[~/lab_programacion]
$ ./ejecutar_lab.sh
```



Para la configuración del cron lo primero que haremos será verificar que el servicio cron este activo.

```
(kali@kali)-[~]
$ sudo systemctl status cron
[sudo] password for kali:
● cron.service - Regular background program processing daemon
   Loaded: loaded (/usr/lib/systemd/system/cron.service; enabled; preset: enabled)
   Active: active (running) since Fri 2026-04-10 03:55:59 EDT; 18min ago
     Invocation: 6d1c45f5e9d2466085d83da00f4aa0926
       Docs: man:cron(8)
    Main PID: 576 (cron)
      Tasks: 1 (limit: 4141)
     Memory: 540K (peak: 2M)
        CPU: 95ms
    CGroup: /system.slice/cron.service
            └─576 /usr/sbin/cron -f

Apr 10 03:55:59 kali systemd[1]: Started cron.service - Regular background program processing daemon.
Apr 10 03:55:59 kali cron[576]: (CRON) INFO (pidfile fd = 3)
Apr 10 03:55:59 kali cron[576]: (CRON) INFO (Running @reboot jobs)
Apr 10 04:05:01 kali CRON[6156]: pam_unix(cron:session): session opened for user root(uid=0) by root(uid=0)
Apr 10 04:05:01 kali CRON[6158]: (root) CMD (command -v debian-sa1 > /dev/null && debian-sa1 1 1)
Apr 10 04:05:01 kali CRON[6156]: pam_unix(cron:session): session closed for user root
Apr 10 04:09:01 kali CRON[8230]: pam_unix(cron:session): session opened for user root(uid=0) by root(uid=0)
Apr 10 04:09:01 kali CRON[8232]: (root) CMD ( [ -x /usr/lib/php/sessionclean ] && if [ ! -d /run/systemd/system ]; then /usr/lib/php/sessionclean; fi)
Apr 10 04:09:01 kali CRON[8230]: pam_unix(cron:session): session closed for user root
```

Lo siguiente será abrir `crontab` para la edición como es la primera vez que lo hacemos en esta máquina, nos aparecerá una lista para escoger el editor, escogernos la primera opción.

```
(kali@kali)-[~]
$ crontab -e
no crontab for kali - using an empty one
Select an editor. To change later, run select-editor again.
 1. /bin/nano          ← easiest
 2. /usr/bin/vim.basic
 3. /usr/bin/vim.tiny

Choose 1-3 [1]: 1
```

Cuando se nos abra el `crontab` lo configuraremos para que se ejecute cada 5 minutos y verificaremos el funcionamiento del `cron`.

```
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
# Ejecutar el programa cada 5 minutos
*/5 * * * * /home/kali/lab_programacion/ejecutar_lab.sh
```



```

(kali@kali)-[~]
$ crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
# Ejecutar el programa cada 5 minutos
*/5 * * * * /home/kali/lab_programacion/ejecutar_lab.sh

```

Y por último esperamos para verificar que se ejecutó de forma correcta.

The image shows a terminal window on the left and a web application on the right. The terminal displays the output of a cron job and the use of the 'tail' command to monitor the output of a script. The web application, titled 'Sistema de Gestión de Calificaciones y Encriptación', displays a selection screen with two options: 'Problema 1 Gestión de Calificaciones' and 'Problema 2 Encriptación de Archivos'.

Terminal Output:

```

(kali@kali)-[~]
$ tail -f ~/lab_program
[2026-04-10 04:11:17] ==
[2026-04-10 04:11:17] ERR
[2026-04-10 04:12:43] ==
[2026-04-10 04:12:43] Ini
[2026-04-10 04:12:43] ==
[2026-04-10 04:12:43] Eje
[2026-04-10 04:14:00] Pro
[2026-04-10 04:14:00] ==
[2026-04-10 04:14:00] Fin
[2026-04-10 04:14:00] ==
[2026-04-10 04:25:01] ==
[2026-04-10 04:25:01] Ini
[2026-04-10 04:25:01] ==
[2026-04-10 04:25:01] Eje
[2026-04-10 04:26:52] Pro
[2026-04-10 04:26:52] ==
[2026-04-10 04:26:52] Fin
[2026-04-10 04:26:52] ==
[2026-04-10 04:30:01] ==
[2026-04-10 04:30:01] Ini
[2026-04-10 04:30:01] ==
[2026-04-10 04:30:01] Eje

```

Web Application Interface:

Sistema de Gestión de Calificaciones y Encriptación

Seleccione una opción

Problema 1
Gestión de Calificaciones

Problema 2
Encriptación de Archivos

Seleccione el problema que desea ejecutar

```
(kali㉿kali)-[~]  
$ tail -f ~/lab_programacion/ejecucion.log  
[2026-04-10 04:11:17] =====  
[2026-04-10 04:11:17] ERROR: No se encuentra el archivo programa_prog.py  
[2026-04-10 04:12:43] =====  
[2026-04-10 04:12:43] Iniciando ejecución del programa de laboratorio  
[2026-04-10 04:12:43] =====  
[2026-04-10 04:12:43] Ejecutando programa_pro.py ...  
[2026-04-10 04:14:00] Programa ejecutado exitosamente  
[2026-04-10 04:14:00] =====  
[2026-04-10 04:14:00] Finalizando ejecución  
[2026-04-10 04:14:00] =====  
[2026-04-10 04:25:01] =====  
[2026-04-10 04:25:01] Iniciando ejecución del programa de laboratorio  
[2026-04-10 04:25:01] =====  
[2026-04-10 04:25:01] Ejecutando programa_pro.py ...  
[2026-04-10 04:26:52] Programa ejecutado exitosamente  
[2026-04-10 04:26:52] =====  
[2026-04-10 04:26:52] Finalizando ejecución  
[2026-04-10 04:26:52] =====  
[2026-04-10 04:30:01] =====  
[2026-04-10 04:30:01] Iniciando ejecución del programa de laboratorio  
[2026-04-10 04:30:01] =====  
[2026-04-10 04:30:01] Ejecutando programa_pro.py ...  
[2026-04-10 04:31:51] Programa ejecutado exitosamente  
[2026-04-10 04:31:51] =====  
[2026-04-10 04:31:51] Finalizando ejecución  
[2026-04-10 04:31:51] =====
```